Markov Chain - Complete Previous-Year Exam Guide

This is the all-in-one Markov sheet: basics + the previous-year patterns we practiced + reversibility + estimating a transition matrix from data + simulation.

1. EXAM RECOGNITION MAP

Read the wording first.

```
One-step probability i -> j
-> P[i,j]

In state j after exactly k transitions
-> P^k

Distribution after k transitions
-> pi0 @ P^k

Absorbing state
-> P[i,i] = 1 and rest of row i = 0

Expected steps until target
-> solve((I-Q)t = 1)

First hit target exactly at step k
-> Q^(k-1) @ r

Never hit target
-> P(T = infinity)

Period / aperiodic
-> possible return times; GCD = 1 means aperiodic

Irreducible
-> every state can eventually reach every other state

Stationary distribution
-> pi @ P = pi and sum(pi)=1

Reversible / detailed balance
-> pi[i]P[i,j] = pi[j]P[j,i]

Hit success before failure
-> solve((I-Q)h = b)

Observed source/destination transitions
-> count transitions and row-normalize to estimate P

Most likely next state
-> np.argmax(P[current])

Simulate next state
-> np.random.choice(..., p=P[current])

Expected value with respect to stationary distribution
-> pi @ state_values
```

2. TRANSITION MATRIX BASICS

For transition matrix P:

• row = starting/current state

• column = next/end state

```
prob = P[start, end]
```

Example:

```
import numpy as np

P = np.array([
```

```
    [0.7, 0.2, 0.1],
    [0.3, 0.4, 0.3],
    [0.2, 0.3, 0.5]
])

prob = P[0, 2]
```

Memory: ROW = START, COLUMN = END.

3. AFTER EXACTLY k TRANSITIONS

Question wording:

> Starting from state i, what is the probability of being in state j after exactly k transitions?

```
Pk = np.linalg.matrix_power(P, k)
prob = Pk[i, j]
```

Example:

```
P5 = np.linalg.matrix_power(P, 5)
prob = P5[1, 0]
```

If an initial distribution is given:

```
Pk = np.linalg.matrix_power(P, k)
pik = pi0 @ Pk
```

Example:

```
P7 = np.linalg.matrix_power(P, 7)
pi7 = pi0 @ P7
```

IMPORTANT:

```
"in D after k steps" -> P^k
```

This does not mean first hitting D at step k.

4. ABSORBING STATES

A state is absorbing if once entered, it cannot be left.

```
P = np.array([
    [0.5, 0.5, 0.0],
    [0.2, 0.5, 0.3],
    [0.0, 0.0, 1.0]
])
```

State 2 is absorbing because its row is:

```
[0, 0, 1]
```

Recognition:

```
P[i, i] == 1
```

and all other entries in row i are zero.

5. Q MATRIX

Q contains transitions only among transient/non-target states.

If states 0 and 1 are transient and state 2 is absorbing:

```
Q = P[:2, :2]
```

If states 0,1,2 are transient and state 3 is absorbing:

```
Q = P[:3, :3]
```

This slicing assumes transient states are listed first.

6. EXPECTED HITTING TIME

Question wording:

> expected number of transitions until the target is reached

Use:

```
Q = P[:2, :2]

expected_steps = np.linalg.solve(
    np.eye(len(Q)) - Q,
    np.ones(len(Q))
)
```

Starting from transient state 1:

```
answer = expected_steps[1]
```

Equivalent fundamental-matrix version:

```
I = np.eye(len(Q))
N = np.linalg.inv(I - Q)
ones = np.ones(len(Q))
expected_steps = N @ ones
```

Memory:

```
EXPECTED TIME -> right-hand side = ONES
(I-Q)t = 1
```

7. FIRST HITTING TIME

Question wording:

> probability of reaching D for the FIRST time exactly at step k

Let Q describe transitions outside D and r describe transitions directly into D.

```
Q = P[:2, :2]
r = P[:2, 2]
```

Then:

```
p_Tk = (
    np.linalg.matrix_power(Q, k - 1) @ r
)[start_index]
```

Example: first hit at step 5 starting from transient state 0:

```
p_T5 = (
    np.linalg.matrix_power(Q, 4) @ r
)[0]
```

Memory:

```
FIRST HIT AT k -> Q^(k-1) @ r
```

Why k-1? Stay outside the target for k-1 transitions, then enter it.

8. P(T = infinity)

This means probability of never reaching the target.

```
start = np.array([1.0, 0.0])

P_T_inf_approx = np.sum(
    start @ np.linalg.matrix_power(Q, 1000)
)
```

If this tends to zero, the target is eventually reached with probability 1.

If the chain can enter another closed/absorbing class that cannot reach the target, P(T=infinity) may be positive.

9. PERIOD AND APERIODICITY

The period of state i is the GCD of possible positive return times to i.

Example:

```
P = np.array([
    [0, 1],
    [1, 0]
])
```

Return times are 2,4,6,... so period = 2.

Fast self-loop shortcut

If:

```
P[i, i] > 0
```

then return in 1 step is possible, so period = 1 and that state is aperiodic.

Do not confuse the probability with the period:

```
P[1,1] = 0.2
0.2 = self-loop probability
1 = return time
period = 1
```

Exam-ready aperiodicity code

```
from math import gcd

def state_period(P, state, max_steps=100):
    return_times = []

    for k in range(1, max_steps + 1):
        Pk = np.linalg.matrix_power(P, k)

        if Pk[state, state] > 1e-12:
            return_times.append(k)

    if len(return_times) == 0:
        return None
```

```
        d = return_times[0]

        for k in return_times[1:]:
            d = gcd(d, k)

        return d
```

Use:

```
period = state_period(P, state=0)
print("Period:", period)
print("Aperiodic:", period == 1)
```

For all states:

```
for i in range(len(P)):
    period = state_period(P, i)
    print("State", i, "period =", period)
```

If the chain is already known to be irreducible and any state has a self-loop:

```
if np.any(np.diag(P) > 0):
    print("Irreducible chain is aperiodic")
```

Important: a positive diagonal entry proves that particular state has period 1. The whole-chain shortcut above relies on irreducibility.

10. IRREDUCIBLE VS REDUCIBLE

Irreducible means every state can eventually reach every other state.

Example:

```
P = np.array([
    [0.0, 1.0],
    [1.0, 0.0]
])
```

This is irreducible.

If there is an absorbing state that cannot return to other states, the whole chain is reducible.

```
P = np.array([
    [0.5, 0.5, 0.0],
    [0.3, 0.4, 0.3],
    [0.0, 0.0, 1.0]
])
```

State 2 cannot return to states 0 or 1, so the whole chain is reducible.

Memory:

```
IRREDUCIBLE -> all states communicate
REDUCIBLE   -> at least one direction/class is cut off
```

11. STATIONARY DISTRIBUTION

A stationary distribution pi satisfies:

```
pi @ P = pi
sum(pi) = 1
```

Exam code:

```
eigvals, eigvecs = np.linalg.eig(P.T)
```

```
idx = np.argmin(np.abs(eigvals - 1))

pi = np.real(eigvecs[:, idx])

pi = pi / np.sum(pi)

print("Stationary distribution:", pi)
```

Check:

```
print(pi @ P)
print(pi)
```

They should be approximately equal.

Memory:

```
P.T
-> eigenvalue closest to 1
-> corresponding eigenvector
-> real part
-> normalize
-> pi
```

12. REVERSIBILITY - PREVIOUS-YEAR PATTERN

Exam wording:

> Is the Markov chain reversible?

or:

> Check detailed balance.

Immediately think:

```
pi[i] P[i,j] = pi[j] P[j,i]
```

First find stationary pi:

```
eigvals, eigvecs = np.linalg.eig(P.T)
idx = np.argmin(np.abs(eigvals - 1))
pi = np.real(eigvecs[:, idx])
pi = pi / np.sum(pi)
```

Then check detailed balance:

```
reversible = True

for i in range(len(P)):
    for j in range(len(P)):

        if not np.isclose(
            pi[i] * P[i, j],
            pi[j] * P[j, i]
        ):
            reversible = False

print("Reversible:", reversible)
```

Meaning:

```
pi[i] * P[i,j] = long-run flow i -> j
pi[j] * P[j,i] = long-run flow j -> i
```

For reversibility, forward and reverse flow must match for every pair.

Why `np.isclose()`? Floating-point calculations may differ by tiny numerical amounts, so use approximate equality instead of `==`.

Memory:

```
REVERSIBLE
-> find stationary pi
-> check detailed balance
-> all pairs equal
-> reversible
```

13. HIT SUCCESS BEFORE FAILURE

Question:

> What is the probability of hitting B before C?

Set boundary values:

```
h[B] = 1   success
h[C] = 0   failure
```

For unknown states:

```
h[i] = sum_j P[i,j]h[j]
```

If states 0 and 1 are unknown, state 2 is success, state 3 is failure:

```
Q = P[:2, :2]
b = P[:2, 2]

h = np.linalg.solve(
    np.eye(len(Q)) - Q,
    b
)
```

Memory:

```
EXPECTED TIME        -> RHS = ones
HITTING PROBABILITY  -> RHS = b
```

14. ESTIMATE TRANSITION MATRIX FROM DATA - PREVIOUS-YEAR PATTERN

This is different from questions where P is already given.

Exam wording may say:

> Each row is an observed transition from a source state/page to a destination state/page.

> Estimate the transition matrix / maximum-likelihood estimate of P.

Think:

```
count source -> destination
then divide each row by its total
```

Example data:

```
source   destination
0        1
0        2
0        2
```

```
1       0
1       2
```

Here 0 -> 1 happened once and 0 -> 2 happened twice. Therefore the estimated probabilities from state 0 are 1/3 and 2/3.

Full code

```
import numpy as np
import pandas as pd

df = pd.read_csv("websites.csv")

source = df["source"].to_numpy(dtype=int)
destination = df["destination"].to_numpy(dtype=int)

n_states = max(
    source.max(),
    destination.max()
) + 1

counts = np.zeros(
    (n_states, n_states),
    dtype=float
)

for s, d in zip(source, destination):
    counts[s, d] += 1

row_totals = counts.sum(axis=1, keepdims=True)

P = np.divide(
    counts,
    row_totals,
    out=np.zeros_like(counts),
    where=row_totals != 0
)

print("Counts:")
print(counts)

print("Estimated transition matrix:")
print(P)
```

Why `+1` for n_states? If state labels are 0,1,2, the largest label is 2 but there are 3 states.

Why row-normalize? A row describes probabilities of leaving one source state, so each nonempty row should sum to 1.

Core memory code

```
for s, d in zip(source, destination):
    counts[s, d] += 1

P = counts / counts.sum(axis=1, keepdims=True)
```

The safer `np.divide` version above avoids division-by-zero warnings if a state has no observed outgoing transitions.

15. MOST LIKELY NEXT STATE

If the current state is 1:

```
current = 1
most_likely = np.argmax(P[current])
```

This returns the index with the largest transition probability.

Two most likely next states:

```
top_two = np.argsort(P[current])[-2:][::-1]
```

Memory:

```
argmax -> position of largest value
argmin -> position of smallest value
```

16. SIMULATE NEXT STATES

Exam wording:

> Simulate 10,000 next states/pages according to the transition probabilities from the current state.

```
current = 1

next_states = np.random.choice(
    n_states,
    size=10000,
    p=P[current]
)
```

Read it as:

```
choose among n_states
10,000 times
using probabilities in row P[current]
```

17. EXPONENTIAL LOAD-TIME TRAP

If the course writes Exp(lambda) using lambda as RATE:

```
mean = 1 / lambda
```

NumPy uses `scale = mean`, not rate.

Therefore:

```
Exp(rate=1)  -> mean 1   -> scale=1.0
Exp(rate=10) -> mean 0.1 -> scale=0.1
```

Code:

```
# Not preloaded: Exp(rate=1)
slow = np.random.exponential(scale=1.0)

# Preloaded: Exp(rate=10)
fast = np.random.exponential(scale=0.1)
```

Do not use `scale=10` when 10 is the rate.

18. SIMULATE PRELOADING THE MOST LIKELY PAGE

```
current = 1
probs = P[current]
most_likely = np.argmax(probs)

next_states = np.random.choice(
    n_states,
    size=10000,
    p=probs
)

load_times = np.empty(10000)

for i, next_state in enumerate(next_states):

    if next_state == most_likely:
        # preloaded: Exp(rate=10), mean=0.1
        load_times[i] = np.random.exponential(
```

```
            scale=0.1
        )
    else:
        # not preloaded: Exp(rate=1), mean=1
        load_times[i] = np.random.exponential(
            scale=1.0
        )

empirical_mean = np.mean(load_times)
print("Empirical mean load time:", empirical_mean)
```

19. PRELOAD TWO MOST LIKELY STATES

```
current = 1

top_two = np.argsort(
    P[current]
)[-2:][::-1]

next_states = np.random.choice(
    n_states,
    size=10000,
    p=P[current]
)

load_times_two = np.empty(10000)

for i, next_state in enumerate(next_states):

    if next_state in top_two:
        load_times_two[i] = np.random.exponential(
            scale=0.1
        )
    else:
        load_times_two[i] = np.random.exponential(
            scale=1.0
        )

print(np.mean(load_times_two))
```

20. THEORETICAL EXPECTED LOAD TIME

If one page is preloaded:

```
current = 1
p_preloaded = np.max(P[current])

expected_load = (
    p_preloaded * 0.1
    + (1 - p_preloaded) * 1.0
)
```

Meaning:

```
probability preload is correct * fast mean
+
probability preload is wrong * slow mean
```

For two preloaded pages:

```
top_two_probs = np.sort(P[current])[-2:]
p_preloaded_two = np.sum(top_two_probs)

expected_load_two = (
    p_preloaded_two * 0.1
    + (1 - p_preloaded_two) * 1.0
)
```

No preloading, if every page has Exp(rate=1):

```
expected_no_preload = 1.0
```

If preload expected/empirical time is lower than 1.0, it improves average load time.

21. STATIONARY DISTRIBUTION + LONG-RUN EXPECTED VALUE

First find pi:

```
eigvals, eigvecs = np.linalg.eig(P.T)
idx = np.argmin(np.abs(eigvals - 1))
pi = np.real(eigvecs[:, idx])
pi = pi / np.sum(pi)
```

Now calculate expected load time for each current state:

```
state_expected_load = []

for i in range(n_states):
    p_best = np.max(P[i])

    e = (
        p_best * 0.1
        + (1 - p_best) * 1.0
    )

    state_expected_load.append(e)

state_expected_load = np.array(
    state_expected_load
)
```

Then weight by stationary probabilities:

```
avg_stationary = pi @ state_expected_load

print(
    "Long-run expected load time:",
    avg_stationary
)
```

Recognition:

```
"expected value with respect to stationary distribution"
-> calculate one value for each state
-> pi @ state_values
```

22. COMPLETE PREVIOUS-YEAR-STYLE DATA TEMPLATE

```
import numpy as np
import pandas as pd

# -------------------------
# 1. Load transition data
# -------------------------
df = pd.read_csv("websites.csv")

source = df["source"].to_numpy(dtype=int)
destination = df["destination"].to_numpy(dtype=int)

# -------------------------
# 2. Estimate P
# -------------------------
n_states = max(source.max(), destination.max()) + 1

counts = np.zeros((n_states, n_states), dtype=float)

for s, d in zip(source, destination):
    counts[s, d] += 1

row_totals = counts.sum(axis=1, keepdims=True)

P = np.divide(
    counts,
```

```
    row_totals,
    out=np.zeros_like(counts),
    where=row_totals != 0
)

# -------------------------
# 3. Most likely next page
# -------------------------
current = 1
most_likely = np.argmax(P[current])

# -------------------------
# 4. Simulate next pages
# -------------------------
next_states = np.random.choice(
    n_states,
    size=10000,
    p=P[current]
)

# -------------------------
# 5. Simulate load times
# -------------------------
load_times = np.empty(10000)

for i, next_state in enumerate(next_states):
    if next_state == most_likely:
        load_times[i] = np.random.exponential(scale=0.1)
    else:
        load_times[i] = np.random.exponential(scale=1.0)

empirical_mean = np.mean(load_times)

# -------------------------
# 6. Theoretical mean
# -------------------------
p_best = np.max(P[current])

theoretical_mean = (
    p_best * 0.1
    + (1 - p_best) * 1.0
)

# -------------------------
# 7. Stationary distribution
# -------------------------
eigvals, eigvecs = np.linalg.eig(P.T)
idx = np.argmin(np.abs(eigvals - 1))
pi = np.real(eigvecs[:, idx])
pi = pi / np.sum(pi)

print("P:", P)
print("Most likely:", most_likely)
print("Empirical mean:", empirical_mean)
print("Theoretical mean:", theoretical_mean)
print("Stationary pi:", pi)
```

23. COMMON EXAM MISTAKES

1. `P[i,j]`: i is start, j is end.
2. "after k steps" -> use `P^k`, not first-hitting code.
3. "first time at k" -> use `Q^(k-1) @ r`.
4. "expected steps until" -> solve `(I-Q)t=1`.
5. Q contains transient/non-target states, not the absorbing target.
6. Self-loop probability is not the period. Any positive self-loop gives return time 1.
7. For stationary pi, normalize so its entries sum to 1.
8. Reversibility requires detailed balance, not just stationarity.
9. Source/destination data gives counts first; row-normalize to obtain probabilities.
10. `np.argmax` gives the index of the largest probability.
11. `np.random.choice(..., p=P[current])` simulates according to transition probabilities.
12. NumPy exponential uses scale/mean. If lambda is a rate, use `scale=1/lambda`.

13. Expected value is a probability-weighted average.

14. Long-run expected quantity -> `pi @ state_values`.

24. LAST-MINUTE EXAM CODE SHEET

k-step probability

```
Pk = np.linalg.matrix_power(P, k)
prob = Pk[start, end]
```

Distribution after k

```
pik = pi0 @ np.linalg.matrix_power(P, k)
```

Expected hitting time

```
t = np.linalg.solve(
    np.eye(len(Q)) - Q,
    np.ones(len(Q))
)
```

First hit exactly at k

```
p_Tk = (
    np.linalg.matrix_power(Q, k - 1) @ r
)[start_index]
```

Hit success before failure

```
h = np.linalg.solve(
    np.eye(len(Q)) - Q,
    b
)
```

Stationary distribution

```
eigvals, eigvecs = np.linalg.eig(P.T)
idx = np.argmin(np.abs(eigvals - 1))
pi = np.real(eigvecs[:, idx])
pi = pi / np.sum(pi)
```

Reversible

```
reversible = True

for i in range(len(P)):
    for j in range(len(P)):
        if not np.isclose(
            pi[i] * P[i,j],
            pi[j] * P[j,i]
        ):
            reversible = False
```

Aperiodic

```
period = state_period(P, state=0)
aperiodic = (period == 1)
```

Estimate P from source/destination data

```
for s, d in zip(source, destination):
    counts[s, d] += 1

P = counts / counts.sum(
    axis=1,
    keepdims=True
)
```

Most likely next state

```
most_likely = np.argmax(P[current])
```

Simulate next state

```
next_states = np.random.choice(
    n_states,
    size=10000,
    p=P[current]
)
```

Long-run expected value

```
answer = pi @ state_values
```

FINAL MEMORY MAP

```
P[i,j]
= one step i -> j

P^k
= state after exactly k transitions

pi0 @ P^k
= distribution after k transitions

(I-Q)t = 1
= expected hitting time

Q^(k-1) @ r
= first hit exactly at k

P(T=infinity)
= never hit target

GCD return times
= period

period 1
= aperiodic

all states communicate
= irreducible

pi @ P = pi
= stationary distribution

pi[i]P[i,j] = pi[j]P[j,i]
= reversible / detailed balance

(I-Q)h = b
= hit success before failure

count source -> destination + row normalize
= estimate transition matrix

argmax(P[current])
= most likely next state

random.choice(..., p=P[current])
= simulate next state
```

```
pi @ state_values
= long-run expected quantity
```

THE 4-LINE EXAM LIFESAVER

```
"after exactly k steps"        -> P^k
"first time exactly at step k" -> Q^(k-1) @ r
"expected steps until"         -> solve(I-Q, ones)
"reversible"                    -> detailed balance using pi
```